

Construction Replay Transcript

Triggered FTP Download · 2026-08-12 · what you'll hear in documents/construction-replay.mp4

What this interface does

Triggered-ftp-download is a PilotFish Sandbox interface. See DESIGN.md for the working purpose.

Who it's for

- Anyone watching the construction video or reading this transcript

Documents we also produce

Capability Brief PDF (generate with tools/export_stakeholder_brief.py) · generate when ready

Short overview of what the interface does — shareable without reading route XML.

Test Plan PDF (from tests/plan.json) · generate when ready

Scenarios that prove the interface works end-to-end.

construction-replay.mp4 + this transcript · ready

Narrated walkthrough of building the routes, then a live Demo-tab test when inject is available.

Narration

Spoken lines from the construction video (setup, build-replay, live test, closing).

Setup

Welcome

Welcome to the PilotFish demo: Triggered FTP Download. A control file names a remote file, then we trigger an FTP listener to go get it. PilotFish supports the whole flow out of the box.

Pipeline overview

Before we wire anything, here's the flow end to end. A control file names a remote file, then we trigger an FTP listener to go get it.

External system

And here's what sits around the routes. Demo Web UI. That's spun up in a docker image for this demo.

01-listen

First route

A control file names a remote file, then we trigger an FTP listener to go get it. Starting with a blank canvas on the first route.

Control-file listener

First up is a directory listener — it watches a local folder for control files. That folder is /opt/pilotfish/input/control. We're only taking .ctl files. It checks about every 5 seconds. After pickup we move the file aside so the folder stays clean.

Stash the file name

The control file is just a name on one line. This processor copies that text onto the transaction so the later steps can use it without parsing the body again.

Remote file name

We trim the name and keep it on the transaction as the remote file name, so we don't carry stray whitespace into FTP.

Remote path

Then we build the full remote path — the upload folder plus that file name — so the download listener knows exactly what to fetch.

Download trigger

Now we kick the FTP download listener for one cycle. This route doesn't pull the file itself — it just tells that listener to go get the named file.

No outbound

This route has nothing to send outbound. The work was the trigger, so we end on a null transport.

Closing

Demo complete

That's the demo — a control file names a remote file, then we trigger an FTP listener to go get it. Thanks for choosing PilotFish.